Develop a Voice Live agent

Speech-capable AI agents enable users to interact conversationally - using spoken command and questions that generate vocal responses.

In this exercise, you'll use the Voice Live capability of Azure Speech in Microsoft Foundry Tools to create a real-time voice-based agent.

This exercise takes approximately **30** minutes.

> **Note**: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active Azure subscription
- Visual Studio Code installed
- Python version **3.13.xx** installed*
- Git installed and configured
- Azure CLI installed

> * Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open Microsoft Foundry at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.

2. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:

    - **Foundry resource**: *Enter a valid name for your AI Foundry resource.*
    - **Subscription**: *Your Azure subscription*
    - **Resource group**: *Create or select a resource group*
    - **Region**: Select any available region

    > **Note**: Use a recommended Microsoft Foundry region. Model and feature availability may vary by region.

3. Select **Create**. Wait for your project to be created. Then view its home page.

Create an agent

Now let's create an agent.

1. Now you're ready to **Start building**. Select **Create agents** (or on the **Build** page, select the **Agents** tab); and create a new agent named `chat-agent`.

When ready, your agent opens in the agent playground.

2. In the model drop-down list, ensure that a **gpt-5** model has been deployed and selected for your agent.

3. Assign your agent the following **Instructions**:

```
You are an AI assistant that helps people find information about AI and related topics. You answer questions concisely and precisely.
```

4. Use the **Save** button to save the changes.

5. Test the agent by entering the following prompt in the **Chat** pane:

```
What can you help me with?
```

The agent should respond with an appropriate answer based on its instructions.

Configure Azure Speech Voice Live

Enabling speech mode for a Foundry agent integrates Azure Speech Voice Live - adding speech capabilities to the agent.

1. In the pane on the left, under the model selection list, enable **Voice mode**.

   If the **Configuration** pane does not open automatically, use the "cog" icon above the chat interface to open it.

2. In the **Configuration** pane, under **Voice Live**, review the default speech input and output configuration. You can try different voices, previewing them until you decide which one to use.

3. Close the **Configuration** pane and use the **Save** button to save the agent.

Use speech to interact with the agent

Now you're ready to chat with the agent.

1. In the Chat pane, use the **Start session** button to start a conversation with the agent. If prompted, allow access to the system microphone.

   The agent will start a speech session, and listen for your prompt.

2. When the app status is **Listening...**, say something like "*How does speech recognition work?*" and wait for a response.

3. Verify that the app status changes to **Processing...**. The app will process the spoken input.

   > **Tip**: The processing speed may be so fast that you do not actually see the status before it changes back to *Speaking*.

4. When the status changes to **Speaking...**, the app uses text-to-speech to vocalize the response from the model. To see the original prompt and the response as text, select the **cc** button on the bottom of the chat screen.

> **Tip**: The follow-on prompt is submitted just by speaking. You can even interrupt the agent to keep the interaction focused on what you need done. You can also use the **Stop generation** button in the chat pane to stop long-running responses. The button will end the conversation. You will need to start a new conversation to continue using the agent.

5. To continue the conversation, just ask another question, such as "*How does speech synthesis work?*", and review the response.
6. When you have finished chatting with the agent, use the **X** icon to end the session. A transcript of the conversation will be displayed.

Create a client application

To use your agent in a custom application, you need to write code that uses the Azure Speech Voice Live SDK to initiate and manage a conversation session.

Get the application files from GitHub

1. Open Visual Studio Code.
2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-language` repo to a local folder (it doesn't matter which one). Then open it.

   You may be prompted to confirm you trust the authors.
3. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at **/Labfiles/06-voice-live/Python/chat-client**. The application files include:
   - **.env** (the application configuration file)
   - **requirements.txt** (the Python package dependencies that need to be installed)
   - **chat-client.py** (the code file for the application)

Configure the application

1. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
2. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.13.xx installation.

> **Tip**: If you are prompted to install dependencies, you can install the ones in the *requirements.txt* file in the */Labfiles/06-voice-live/Python/chat-client* folder; but it's OK if you - don't we'll install them later!

> **Tip**: If you prefer to use the terminal, you can create your **Venv** environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.

3. In the **Explorer** pane, right-click the **chat-client** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */Labfiles/06-voice-live/Python/chat-client* folder.)

> **Note**: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

4. Ensure that the terminal is open in the **chat-client** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.

5. Install the Foundry SDK package, the Azure Identity package, and other required packages by running the following command:

```
pip install -r requirements.txt azure-identity azure-ai-voicelive==1.2.0b4 --pre azure-ai-projects==2.0.0b4
```

6. In the **Explorer** pane, in the **chat-client** folder, select the **.env** file to open it. Then update the configuration values to include your Foundry resource **endpoint** (get the project endpoint from the project home page in Foundry Portal, but use only the base URL up to the *.com* domain), your project name, and the name of your agent (which should be **Chat-Agent** - note that this name is case-sensitive).

> **Important**: Modify the pasted endpoint to remove the "/api/projects/{project_name}" suffix - the endpoint should be *https://{your-foundry-resource-name}.services.ai.azure.com*.

1. Save the modified configuration file.

Implement application code

1. In the **Explorer** pane, in the **chat-client** folder, open the **chat-client.py** file.

2. Review the existing code. Most of the application scaffolding has been provided - you must implement the key steps required to use the Voice Live SDK to manage a conversation with your agent.

> **Tip**: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespaces you will need:

```
# import namespaces
from azure.identity.aio import AzureCliCredential
from azure.ai.voicelive.aio import connect
from azure.ai.voicelive.models import (
    InputAudioFormat,
    Modality,
    OutputAudioFormat,
    RequestSession,
    ServerEventType,
    AudioNoiseReduction,
    AudioEchoCancellation,
    AzureSemanticVadMultilingual,
    AgentConfig
)
```

4. In the **main** function, note that code to load the endpoint from the configuration file has already been provided, as has code to get an authentication credential and to create and run a **VoiceAssistant** object.

   The **VoiceAssistant** class encapsulates the logic to manage the Voice Live conversation.

5. Under the **main** function, find the **VoiceAssistant** class definition.

   The **init** function to initialize an object based on the class has already been implemented.

   You must implement the **start** function, which is the core function to establish the conversation session.

6. Find the comment **STEP 1: Connect Azure VoiceLive to the agent**, and add the following code (being careful to indent it one level in under the **try:** statement):

Code Copy

```
# STEP 1: Connect Azure VoiceLive to the agent
async with connect(
    endpoint=self.endpoint,
    credential=self.credential,
    api_version="2026-01-01-preview",
    agent_config=self.agent_config
) as connection:
    self.connection = connection
```

This step creates a connection to your agent so the Voice Live SDK can establish a conversation with it.

7. Find the comment **STEP 2: Initialize audio processor**, and add the following code (being careful to indent it *another level in* under the step 1 code you just added):

Code Copy

```
# STEP 2: Initialize audio processor
self.audio_processor = AudioProcessor(connection)
```

This code attaches an AudioProcessor object based on the class definition further down in the code file. The AudioProcessor is a utlility class to manage audio hardware I/O.

8. Find the comment **STEP 3: Configure the session**, and add the following code (being careful to maintain the same indentation as the step 2 code above):

Code Copy

```
# STEP 3: Configure the session
await self.setup_session()
```

This code configures the session with the appropriate audio formats, conversational turn-detection semantics, and options to handle echos and background noise.

9. Find the comment **STEP 4: Start audio systems**, and add the following code (being careful to maintain the same indentation as the step 3 code above):

Code Copy

```
# STEP 4: Start audio systems
self.audio_processor.start_playback()

print("\n✅ Ready! Start speaking...")
print("Press Ctrl+C to exit\n")
```

This code starts the audio processor so that it monitors the microphone for audio input and plays back audio output.

10. Find the comment **STEP 5: Process events**, and add the following code (being careful to maintain the same indentation as the step 4 code above):

Code Copy

```
        # STEP 5: Process events
        await self.process_events()
```

This code runs the main loop to process events such as speech input, response output, and interruptions.

11. Save the changes to the code file.

    The completed function should look like this:

Code Copy

```
async def start(self):
        """Start the voice assistant."""
        print("\n" + "=" *60)
        print(f"🎙️   {self.agent_config['agent_name']}")
        print("="* 60)

        # Add your code in this try block!
        try:
            # STEP 1: Connect Azure VoiceLive to the agent
            async with connect(
                endpoint=self.endpoint,
                credential=self.credential,
                api_version="2026-01-01-preview",
                agent_config=self.agent_config
            ) as connection:
                self.connection = connection

                # STEP 2: Initialize audio processor
                self.audio_processor = AudioProcessor(connection)

                # STEP 3: Configure the session
                await self.setup_session()

                # STEP 4: Start audio systems
                self.audio_processor.start_playback()

                print("\n✅ Ready! Start speaking...")
                print("Press Ctrl+C to exit\n")

                # STEP 5: Process events
                await self.process_events()

        finally:
            if hasattr(self, 'audio_processor'):
                self.audio_processor.shutdown()
```

Run the application

Now you're ready to run your application, and have a conversation with your agent.

> **TIP**: The application works best when using a headset. When using speakers, there's a risk that the agent can "hear" its own responses and process them as new user input.

1. In the Visual Studio Code terminal, enter the following command to sign into Azure

Prerequisites

Create a Microsoft Foundry project

Create an agent

Configure Azure Speech Voice Live

Use speech to interact with the agent

Create a client application

Run the application

Clean up

```
az login
```

When prompted, sign into Azure using your credentials.

> **Note**: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See *Sign into Azure interactively using the Azure CLI* for details.

2. In the Visual Studio Code terminal, confirm the details of your Azure subscription; and then enter the following command to run the client application:

```
python chat-client.py
```

3. When prompted, begin a conversation with the agent by asking a question such as "*How is computer speech used in AI?*".
4. Listen to the response and then continue the conversation - note that you can interrupt the agent to ask new questions.
5. When you're finished, press **CTRL+C** to end the conversation and stop the program.

Clean up

If you have finished exploring Microsoft Foundry, delete any resources that you no longer need. This avoids accruing any unnecessary costs.

1. Open the **Azure portal** at https://portal.azure.com and select the resource group that contains the resources you created.
2. Select **Delete resource group** and then **enter the resource group name** to confirm. The resource group is then deleted.